

---

# Lab Report

## Requirements Specification

---

**Student:** Mouhamadou Taha Thiam

**Program:** Software Engineering

**Supervisor:** Imane Fouad

## Contents

|          |                                             |          |
|----------|---------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                         | <b>2</b> |
| <b>2</b> | <b>Exercise 1: Warming up</b>               | <b>2</b> |
| <b>3</b> | <b>Exercise 2: Train Ticket Distributor</b> | <b>4</b> |
| <b>4</b> | <b>Exercise 3: Recruitment System</b>       | <b>5</b> |
| 4.1      | Functional requirements . . . . .           | 5        |
| 4.2      | Use case diagram . . . . .                  | 5        |
| <b>5</b> | <b>Exercise 4: Hotel Management System</b>  | <b>7</b> |
| 5.1      | Actors of the system . . . . .              | 7        |
| 5.2      | Actions outside the system . . . . .        | 7        |
| 5.3      | System boundaries . . . . .                 | 8        |
| 5.4      | Use case diagram . . . . .                  | 8        |
| <b>6</b> | <b>Conclusion</b>                           | <b>9</b> |

## 1 Introduction

This lab focuses on requirements engineering. Its objectives are to:

- understand how to extract and organize system requirements;
- learn the basics of identifying actors and use cases;
- practice modeling requirements using a use case diagram.

## 2 Exercise 1: Warming up

### Question 1

Assuming a productivity of 100 lines of code per day, how long would it take, working alone, to implement Firefox? And Google? Express the answer in days and years.

From Figure 1 of the subject, Firefox contains about 10 million lines of code, and Google services about 2,000 million lines.

$$\begin{aligned}\text{Firefox: } \frac{10\,000\,000}{100} &= 100\,000 \text{ days} \approx \frac{100\,000}{365} \approx 274 \text{ years} \\ \text{Google: } \frac{2\,000\,000\,000}{100} &= 20\,000\,000 \text{ days} \approx \frac{20\,000\,000}{365} \approx 54\,795 \text{ years}\end{aligned}$$

In other words, implementing Firefox alone would take several lifetimes, and implementing Google would take longer than the whole of recorded human history.

### Question 2

How did these companies succeed despite such enormous amounts of code?

These companies succeed thanks to efficient teamwork: the system is divided into smaller parts, and many teams work on them in parallel. They also reuse existing code and rely on good tools and methods to coordinate their work.

### Question 3

One developer writes 500 lines to solve a problem, another only 50. Who is more productive? Propose better ways to evaluate developer or team productivity.

The most productive developer is the one who solves the problem with only 50 lines, provided the solution works correctly: shorter code is usually simpler, easier to read and easier to maintain. The number of lines of code is therefore a poor measure of productivity.

A better way to evaluate developer or team productivity is to look at the effectiveness of the final product: the features delivered, the quality of the software (few bugs), and the satisfaction of its users.

### Question 4

In a future where AI generates most of the code, what would remain as the unique contribution of human developers?

The unique contribution of human developers would be the idea behind the project: understanding the needs of the users, deciding what the system should do, and verifying that the generated code actually meets these needs.

### 3 Exercise 2: Train Ticket Distributor

#### Statement

Create a use case diagram for a ticket distributor system for trains, based on the given actors and use cases.

The system involves two actors, both located outside the system boundary:

- **Traveler:** the person who buys tickets. They initiate the use cases *BuyOneWayTicket*, *BuyWeeklyCard* and *BuyMonthlyCard*.
- **Central Computer System:** an external system that maintains the reference database. It initiates the use case *UpdateTariff*.

The four use cases are placed inside the boundary of the *Ticket Distributor System*, as shown in Figure 1.

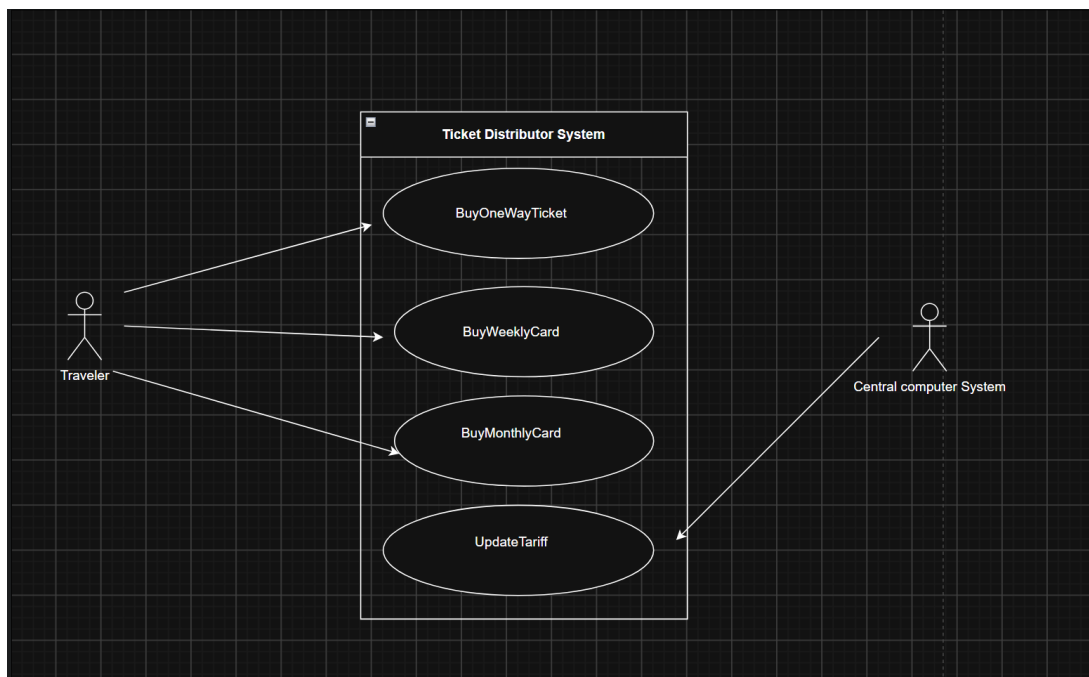


Figure 1: Use case diagram of the ticket distributor system

## 4 Exercise 3: Recruitment System

### 4.1 Functional requirements

#### Question 1

Give 5 functional requirements of the system. For each requirement, indicate its priority and estimated effort.

Table 1 lists five functional requirements of the system. The priority is expressed as *High*, *Medium* or *Low*, and the effort is estimated in person-days.

| ID  | Requirement                                                                          | Priority | Effort  |
|-----|--------------------------------------------------------------------------------------|----------|---------|
| FR1 | The system shall allow the job applicant to create a CV.                             | High     | 10 days |
| FR2 | The system shall allow the career expert to send recommendations to the applicant.   | High     | 4 days  |
| FR3 | The system shall allow the career expert to add a CV to a shortlist.                 | High     | 3 days  |
| FR4 | The system shall allow the client manager to create automated tests.                 | High     | 15 days |
| FR5 | The system shall send weekly job announcements matching the applicants' preferences. | Medium   | 8 days  |

Table 1: Functional requirements of the recruitment system

### 4.2 Use case diagram

#### Question 2

Draw the use case diagram of the system, keeping the modules as described. Highlight each module using system boundaries.

The system is decomposed into three modules, each represented by its own boundary inside the global system:

- **CV Builder:** used by the job applicant to create, modify and resubmit their CV;
- **Orientation:** used by the career expert to review CVs, send recommendations and add validated CVs to a shortlist;
- **Tests:** used by the client manager to create automated tests, send the list of successful candidates and inform unsuccessful ones; the job applicant uses it to take the automated tests.

Two features are placed outside the modules, as they are global to the system: the weekly sending of job announcements, triggered by a **Timer** actor, and the reception of the list of successful candidates by the **Recruiter**. The complete diagram is shown in Figure 2.

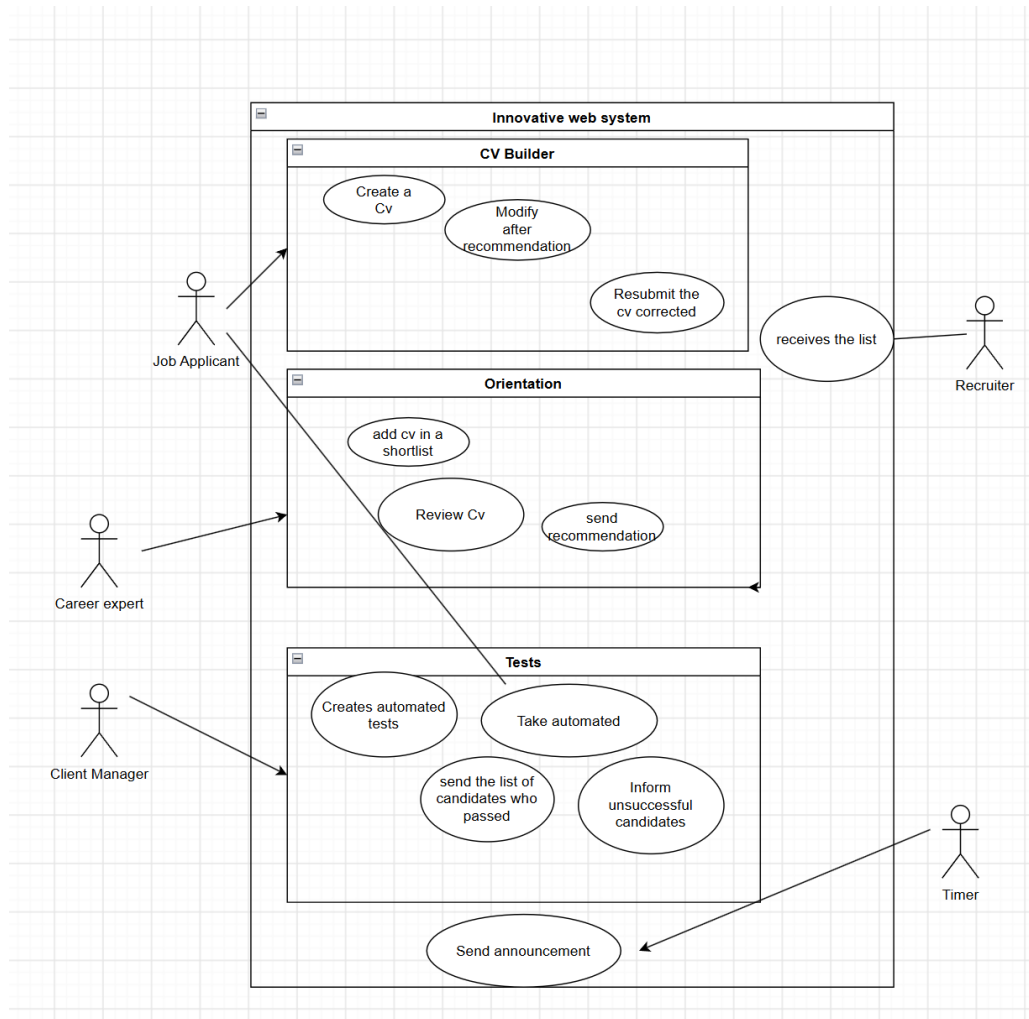


Figure 2: Use case diagram of the recruitment system

## 5 Exercise 4: Hotel Management System

### 5.1 Actors of the system

#### Question 1

Based on the text, identify the actors of the system.

Table 2 lists the actors identified from the description.

| Actor          | Type                        | Role                                                                        |
|----------------|-----------------------------|-----------------------------------------------------------------------------|
| Client         | Primary                     | Makes reservations via the web or mobile app and pays online.               |
| Receptionist   | Primary                     | Searches for rooms, verifies reservations, performs check-in and check-out. |
| Cashier        | Primary                     | Adds consumptions to the client's invoice.                                  |
| Cleaning staff | Primary                     | Marks cleaned rooms as "clean waiting".                                     |
| Floor manager  | Primary                     | Validates cleaning and marks rooms as "available" or "unavailable".         |
| Bank server    | Secondary (external system) | Validates online payments.                                                  |

Table 2: Actors of the hotel management system

The maintenance technician is not an actor: the description never states that they use the application. Their intervention takes place outside the system.

### 5.2 Actions outside the system

#### Question 2

Identify the actions that occur outside the system.

The following actions take place outside the system:

- the reservation made by phone (the call itself does not go through the application);
- the arrival of the client at the hotel;
- the consumption of meals (only their addition to the invoice is done in the system);
- the physical cleaning of the rooms;
- the physical inspection of the rooms by the floor manager;
- the maintenance intervention (air conditioner failure, leaks, etc.);
- the validation of the payment, performed by the external bank server.

Note that the choice of Node.js and Android, as well as the use of SSL, are not actions: they are technical constraints, i.e. non-functional requirements, and therefore do not appear in the use case diagram.

### 5.3 System boundaries

#### Question 3

If we wanted to decompose the system into boundaries, what would these boundaries be?

The system can be decomposed into four functional modules:

1. **Reservation:** online reservation and payment;
2. **Reception:** room search, reservation verification, check-in and check-out;
3. **Billing:** management of invoices and consumptions;
4. **Cleaning:** management of room statuses after the client leaves.

### 5.4 Use case diagram

#### Question 4

Draw the use case diagram of the system with functional boundaries (modules).

The use case diagram, organized according to the four modules above, is shown in Figure 3.

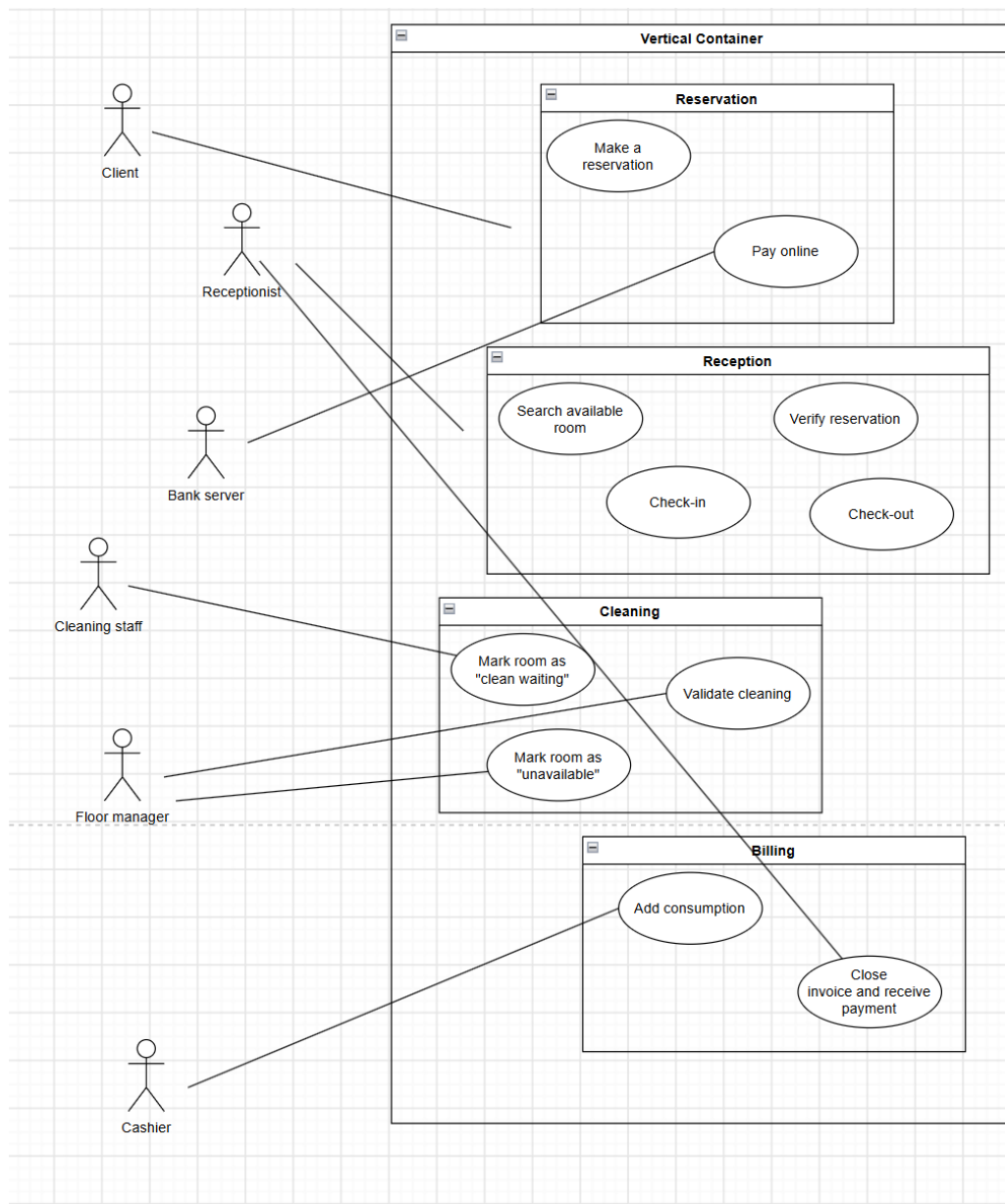


Figure 3: Use case diagram of the hotel management system

## 6 Conclusion

This lab introduced the first steps of requirements engineering. The warm-up exercise showed that modern software systems are far too large to be built by a single person, and that the number of lines of code is a poor measure of productivity. The following exercises allowed us to practice identifying actors, distinguishing what happens inside and outside a system, writing functional requirements with their priority and effort, and modeling them with use case diagrams organized into modules. These skills are essential to understand the needs of users before starting the design and development of a system.